Ved Dalwadi

In this journal entry for the Final project, I will discuss how I used learning from previous modules to apply SMOTE balancing and to train all 6 machine learning models to be able to predict whether a customer will churn or not, and each model's accuracy of doing so. This final project showed me how to actually apply the things I have learned in the course, like SMOTE balancing or capping a decision tree's depth. This also made me learn 2 new models which I only saw in previous PowerPoint presentations.

First, let's talk about why predicting churn matters. Being able to predict when a customer will churn is a great model to train; if the model is accurate enough, your company will be able to hold long term customers who keep spending. This would be better than hiring a marketing team to cold call or advertise their service, since telemarketers only turn a small number of customers they contact. This method of predicting when a customer is about to churn means that you can entice them to stay, which is more profitable than trying to find a new customer. In this project, I have made a classification machine learning model with a 70/15/15 split to preserve a 26.5% churn rate across 3 splits to make it balanced and not have 22% in one and 29% in another. This was built on the data cleaning my group did previously in the midterm.

Keeping the 26.5% churn rate the same across the 3 splits to make sure it was realistic. However, this showed that the training data itself was imbalanced, and the model could always just say no to everything that came at it, and it would get 73.5% accuracy. Back in Module 7, I also stated that accuracy can be misleading; this is also happening here, where the model is hiding behind a high accuracy rate. This previous project let me see this and fix this mistake by using SMOTE on the training data, which lets me create made up samples of churned customers instead of just duplicating.

After I balanced the training data using SMOTE, I went to train all 6 of the models, where I first did logistic regression and finally ended with 2 new ones I have only seen, not worked with. While my code structure stayed the same throughout the models, I knew something was off with the decision tree. At first glance, it seemed like everything was fine, as it was getting around 70% per model, which was the average between all the models. However, I wrote my code so that the tree had unlimited depth. Then, I realized that this model could be overfitting severely, so I put that line of code to check, and I was right: 99% to 70% of models. This meant that my model was overfitting; however, this is a simple fix, as I just chose a depth of 5. This

increased most of the model's accuracy scores, as it's not just remembering now and is learning. It brought the training accuracy down to 77.1%, which is much closer to all the other accuracy scores, which now means it's a good fit. This brought me back to module 1, where I had asked the simple question about why the decision tree can't have more depth. I had researched and been told it's because the model would overfit and just memorize the data. Now that I understand the topic and why it's a major issue, everything seems clearer to me. A model that only memorizes will make bad predictions on new unseen data. Back then, I believed that model memorization wasn't the worst thing, since it still remembers the data, but now that I'm building the model, it makes much more sense why memorization is such a major issue.

After I had trained all 6 models, I was told to use the Voting Classifier to see if the average of the best 3 models would do better than the individual. When I started working with the voting classifier, I noticed how it was similar to a weather app. Where each weather app may tell you a different % chance of rain, one weather app may tell you 80%, another may tell you 70%, and another may tell you 60%. I noticed that a voting classifier would take this information and say that there is a 70% chance of rain. This is what the voting classifier is used for; instead of just trusting one accuracy rating, this would put the trust in multiple and, like in Lab 9 with the wisdom of the crowd, more models are almost always better than just one good model.

In conclusion, this final project has shown me that the methods I learned in labs can have many different applications outside of the one scenario we are using them for as if we train them right, we can have them almost predict anything. This has also taught me how much I have learned from these modules by being able to see things I learned about and worked with 2-3 weeks ago and applying it to something brand new.